CTN — Clone TabNews

PLANO DE GERENCIAMENTO DE CONFIGURAÇÃO

Enfoque DevOps — Integração e Entrega Contínuas (CI/CD)

Versão 1.1

Campo	Descrição
Projeto	CTN — Clone TabNews
Gerente de Projeto	Paulo Guilherme
Gerente/Responsável de GCS-DevOps	Paulo Guilherme
Fábrica de Software / Squad	Squad PG — Paulo Guilherme (squad individual)
Repositório principal	PauloGuilhermee/clone-tabnews: Projeto para estudar programação

Sumário

1. Introdução **3**
1.1 Finalidade 3
1.2 Escopo 3
1.3 Definições, Acrônimos e Abreviações 4
1.5 Visão Geral 4
2. Cultura DevOps e Papéis na Gerência de Configuração **4**
3. Ferramentas, Ambientes e Infraestrutura **5**
3.1 Ferramentas do Pipeline 5
3.2 Ambientes e Infraestrutura 6
3.3 Estratégia de Branching e Versionamento 6
4. Identificação da Configuração **6**
4.1 Itens de Configuração 6
4.2 Convenção de Nomenclatura 7
4.3 Estrutura do Repositório 7
5. Integração Contínua (CI) **8**
5.1 Etapas do Pipeline de CI 8
5.2 Política de Pull Request e Revisão de Código 8
5.3 Critérios de Qualidade (Quality Gate) 8
6. Entrega e Implantação Contínua (CD) **8**
6.1 Etapas do Pipeline de CD 8
6.2 Estratégias de Implantação 9
6.3 Rollback e Recuperação 9
7. Infraestrutura como Código (IaC) **9**
7.1 Versionamento de Infraestrutura 9
7.2 Ambientes Efêmeros 10
8. Controle de Configuração e Mudança **10**
8.1 Processo de Solicitação de Mudança 10
8.2 Comitê de Controle de Mudança (CCM) 10
9. Baselines e Releases **10**
9.1 Geração Automática de Baseline 10
9.2 Notas de Versão (Release Notes) 10
10. Estimativa do Status de Configuração **11**
10.1 Monitoramento e Observabilidade 11
10.2 Métricas DORA 11
10.3 Relatórios e Auditorias 11
11. Marcos **11**
12. Treinamento e Recursos **12**
13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros **12**
14. Quadro-Resumo do Pipeline CI/CD **13**

1. Introdução

Este Plano de Gerenciamento de Configuração (PGC) descreve as atividades de Gestão de Configuração de Software (GCS) que serão executadas ao longo do ciclo de vida do produto, com ênfase em práticas DevOps — em particular Integração Contínua (CI), Entrega/Implantação Contínua (CD), Infraestrutura como Código (IaC) e automação de pipelines. As atividades envolvem identificar os itens de configuração, manter sua integridade e controlar sistematicamente as mudanças, com o mínimo de burocracia possível e o máximo de automação.

1.1 Finalidade

Estabelecer um padrão a ser seguido pela equipe/individuo, garantindo rastreabilidade, versionamento e controle de mudanças do produto, apoiado por pipelines automatizados de build, teste, integração e implantação.

1.2 Escopo

Este PGC é destinado à Squad PG, composta por um único integrante (Paulo Guilherme), que acumula todos os papéis descritos na seção 2. O plano abrange o sistema CTN — Clone TabNews: uma API REST construída com Next.js (API Routes) e PostgreSQL, desenvolvida como projeto de estudo no curso.dev, que implementa cadastro e atualização de usuários, autenticação por e-mail e senha e gestão de sessões via cookie. A interface web é mínima (página inicial e página de status). O CTN é um **projeto de teste, ainda em desenvolvimento e sem usuários reais**; por isso, alguns controles são simplificados e os riscos conhecidos estão registrados explicitamente neste documento (seção 11).

1.3 Definições, Acrônimos e Abreviações

Termo	Significado
GCS	Gestão de Configuração de Software
CI	Integração Contínua (Continuous Integration)
CD	Entrega/Implantação Contínua (Continuous Delivery / Deployment)
IaC	Infraestrutura como Código (Infrastructure as Code)
CCM	Comitê de Controle de Mudanças
Baseline	Conjunto de itens de configuração que atingiram um estado comprovado de estabilidade
Pipeline	Sequência automatizada de etapas de build, teste, análise e implantação
Trunk-based Development	Estratégia de branching em que os desenvolvedores integram na branch principal com alta frequência
SemVer	Versionamento Semântico (MAJOR.MINOR.PATCH) - 1.0.0.
DORA Metrics	Métricas de desempenho DevOps (Lead Time, Deployment Frequency, MTTR, Change Failure Rate)
Preview Deploy	Implantação temporária gerada pela Vercel a cada Pull Request, com URL própria, usada como ambiente de homologação
Neon	Serviço gerenciado de PostgreSQL utilizado nos ambientes de homologação e produção
Migration	Script versionado que altera o esquema do banco de dados (node-pg-migrate)

Termo	Significado
SAST / SCA	Análise estática de segurança do código-fonte / análise de vulnerabilidades em dependências de terceiros
EOL	End of Life — versão de software que deixou de receber suporte e correções de segurança

1.5 Visão Geral

Diferente de processos tradicionais em que a GCS é predominantemente manual e concentrada em marcos formais, neste plano a GCS é tratada como parte constante do pipeline de entrega: cada commit integrado, testado e implantado automaticamente gera evidências de rastreabilidade (logs de pipeline, tags de versão, artefatos publicados), reduzindo o esforço manual e o tempo entre a mudança e sua entrega em produção.

2. Cultura DevOps e Papéis na Gerência de Configuração

A GCS em um contexto **DevOps é responsabilidade compartilhada entre desenvolvimento e operações ("Você constrói, você executa" — you build it, you run it), apoiada por automação.** Os papéis abaixo descrevem responsabilidades típicas, que podem ser exercidas por uma mesma pessoa em squads pequenas. No CTN, todos os papéis abaixo são exercidos por uma única pessoa, Paulo Guilherme.

Papel	Responsabilidades
Gerente de Projeto / Product Owner	Paulo Guilherme. Prioriza o backlog (GitHub Issues), decide o escopo das entregas e acompanha as métricas DORA (seção 10.2).
Engenheiro DevOps / Release Engineer	Paulo Guilherme. Mantém os workflows do GitHub Actions, a integração com a Vercel e o Neon e as convenções de versionamento (Conventional Commits e SemVer).
Desenvolvedor(a)	Paulo Guilherme. Integra código na main por PRs de vida curta, escreve os testes de integração (Jest) e segue Conventional Commits, validados por commitlint e Husky.
SRE / Operações (Site Reliability Engineering)	Paulo Guilherme. Verifica a saúde da aplicação pelo endpoint /api/v1/status e pelos logs da Vercel, aplica migrations após o deploy e executa rollback quando necessário. Não há SLIs/SLOs formais nesta fase, por se tratar de projeto de teste.
QA / Automação de Testes	Paulo Guilherme. Mantém a suíte de testes de integração em tests/integration, executada automaticamente a cada PR.
Segurança (DevSecOps)	Paulo Guilherme. Mantém o secretlint (hook pre-push e CI, bloqueante). SCA e SAST não estão automatizados nesta fase; vulnerabilidades em dependências são verificadas manualmente com npm audit.
Comitê de Controle de Mudanças (CCM)	Composto unicamente por Paulo Guilherme. Acionado apenas nos casos definidos na seção 8.2; as demais mudanças seguem o fluxo de PR + pipeline.

3. Ferramentas, Ambientes e Infraestrutura

3.1 Ferramentas do Pipeline

Ferramentas efetivamente adotadas no projeto CTN:

Categoria	Ferramenta(s) adotada(s)	Finalidade
Controle de versão	Git + GitHub	Versionamento de código-fonte, testes, migrations, pipelines e configurações.
Orquestração de CI/CD	GitHub Actions + integração Git da Vercel	CI a cada PR (lint, commits, segredos, testes); deploy automático pela Vercel.
Padronização de commits	commitlint + Husky + Commitizen	Garantir Conventional Commits (hook commit-msg local e job no CI).
Qualidade de código	ESLint + Prettier + EditorConfig	Análise estática e formatação padronizada.
Testes automatizados	Jest	Testes de integração dos endpoints da API contra banco real em contêiner.
Versionamento	Git + Conventional Commits + SemVer	Histórico rastreável por tipo de mudança; versão declarada no package.json (1.0.0). Tags e releases automáticas ainda não adotadas.
Empacotamento	Build da Vercel	Cada deploy é um artefato imutável e identificado, reutilizável para rollback. Docker não é usado para empacotar a aplicação.
Serviços locais	Docker Compose (PostgreSQL 16 + Mailcatcher)	Subir as dependências do ambiente de desenvolvimento e do CI de forma reprodutível.
Banco de dados e migrations	Neon (PostgreSQL) + node-pg-migrate	Banco de produção e de homologação; esquema versionado por migrations.
Gestão de segredos	Environment Variables da Vercel + secretlint	Segredos de produção fora do repositório; bloqueio de push/PR com credenciais.
Monitoramento	Endpoint /api/v1/status + logs da Vercel + painel do Neon	Verificação de saúde (versão e conexões do banco) e diagnóstico de falhas.
Dependências	package-lock.json	Travamento de versões; verificação de vulnerabilidades feita manualmente (npm audit), fora do pipeline.
Rastreamento de tarefas	GitHub Issues	Vinculação de PRs a itens de trabalho e registro das decisões do CCM.

3.2 Ambientes e Infraestrutura

O CTN utiliza três ambientes. Homologação e Produção são gerenciados pelas plataformas Vercel e Neon; o ambiente local é descrito como código (infra/compose.yaml), garantindo que possa ser recriado de forma rastreável.

Ambiente	Finalidade	Gatilho de promoção
Desenvolvimento (local)	Máquina do desenvolvedor: Next.js em modo dev + PostgreSQL e Mailcatcher via Docker Compose, configurados por .env.development. O mesmo arranjo é usado pelo CI para executar os testes.	Criação de branch feature/… e execução de npm run dev

Ambiente	Finalidade	Gatilho de promoção
Homologação / Preview	Preview Deploy da Vercel gerado a cada PR, com URL própria, conectado a um banco de homologação separado (branch do Neon), compartilhado entre os Previews.	Abertura ou atualização de Pull Request para a main
Produção	Deploy de produção da Vercel, conectado ao banco de produção no Neon.	Merge na main (PR com checks obrigatórios verdes)

Detalhamento: a produção usa o domínio atribuído pela Vercel e cada Preview recebe uma URL única. A topologia é gerenciada pelas plataformas (funções serverless da Vercel acessando o PostgreSQL do Neon), sem balanceador ou servidores administrados pelo projeto. As credenciais de Homologação e Produção (conexão com o banco, PEPPER_PASSWORD e SMTP, quando aplicável) ficam nas Environment Variables da Vercel, separadas por ambiente. O arquivo .env.development contém apenas valores do ambiente local em contêiner. Ele consta no .gitignore, mas foi versionado antes dessa regra e continua rastreado pelo Git, o que permite reproduzir o ambiente e executar o CI; seus valores não são usados em produção.

3.3 Estratégia de Branching e Versionamento

O CTN adota GitHub Flow, uma variação do Trunk-Based Development, com branches de vida curta integradas à main por Pull Request:

- main — sempre implantável; cada merge gera deploy em produção. As mudanças chegam por Pull Request, por convenção; ainda não há regra de proteção de branch configurada no GitHub (pendência registrada na seção 11).
- feature/<descrição-curta> — uma branch de curta duração para cada mudança. Branches antigas (ex.: sessions) não seguiam o prefixo; a convenção vale a partir deste plano.
- release/x.y — não utilizada: a própria main é a linha de release.
- hotfix/<descrição> — correções urgentes em produção, pelo mesmo fluxo de PR; se for preciso contornar alguma regra, a mudança passa pelo CCM (seção 8.2).

Versionamento: Versionamento Semântico (MAJOR.MINOR.PATCH), com a versão declarada no package.json (atualmente 1.0.0), e Conventional Commits validados pelo commitlint. Pela convenção, fix corresponde a PATCH, feat a MINOR e BREAKING CHANGE a MAJOR. A geração automática de tags e releases ainda não está implantada (seção 9).

4. Identificação da Configuração

4.1 Itens de Configuração

Os seguintes artefatos são itens de configuração do CTN, todos versionados no mesmo repositório:

- Código-fonte da aplicação (pages/, models/, infra/*.js) e dos testes automatizados (tests/).
- Pipelines como código: .github/workflows/ (linting.yaml e test.yaml).
- Definição do ambiente local em contêineres: infra/compose.yaml. Não há Dockerfile, pois a aplicação é empacotada pela Vercel.
- Infraestrutura como Código declarativa (Terraform, Ansible etc.): não utilizada; Vercel e Neon são configurados pelos painéis (ver seção 7).
- Arquivos de configuração: .env.development (apenas valores locais), .nvmrc, jsconfig.json, jest.config.js, .eslintrc.json, .prettierrc, .editorconfig, commitlint.config.js, .secretlintrc.json e hooks em .husky/. Segredos de produção ficam na Vercel.

- Scripts de migração de banco de dados: infra/migrations/ (node-pg-migrate).
- Manifesto e travamento de dependências: package.json e package-lock.json.
- Documentação técnica: README.md, LICENSE e este PGCS.

4.2 Convenção de Nomenclatura

Item	Convenção	Exemplo
Branch de feature	feature/<descrição-curta>	feature/password-reset
Commit	Conventional Commits: <tipo>(<escopo opcional>): <descrição>	feat: add `email.js` infra module
Tag de release (planejada)	Versionamento Semântico vX.Y.Z	v1.0.0
Migration	<timestamp>_<descrição-em-kebab-case>.js (npm run migrations:create)	1776638490928_create-sessions.js
Endpoint da API	/api/v<versão>/<recurso>	/api/v1/sessions
Workflow de CI/CD	<finalidade>.yaml em .github/workflows/	test.yaml

4.3 Estrutura do Repositório

Repositório único, com a seguinte estrutura:

```
clone-tabnews/
├── .github/workflows/  → pipelines (linting, test)
├── .husky/             → hooks de Git (commit-msg, pre-push)
├── infra/              → compose.yaml, banco, erros, controller, e-mail
│   ├── migrations/     → migrations versionadas
│   └── scripts/        → orquestração e espera do banco
├── models/             → regras de negócio (user, session, authentication...)
├── pages/              → páginas e rotas do Next.js
│   └── api/v1/         → endpoints REST (users, sessions, status...)
├── tests/integration/  → testes de integração (Jest)
├── package.json, package-lock.json, .nvmrc
└── README.md
```

5. Integração Contínua (CI)

Todo Pull Request para a main dispara automaticamente os workflows de CI no GitHub Actions (linting.yaml e test.yaml). O resultado dos checks aparece no próprio PR, e o merge só é feito com todos verdes.

5.1 Etapas do Pipeline de CI

1. Checkout do código e instalação travada das dependências (npm ci, a partir do package-lock.json), com Node.js 18 definido em .nvmrc.
2. Build: não há job de build dedicado no GitHub Actions; a aplicação é compilada pelo Next.js durante os testes e pela Vercel no Preview Deploy de cada PR.
3. Análise estática e formatação: Prettier e ESLint (linting.yaml).
4. Padrão de commits: commitlint valida todos os commits do PR (linting.yaml).

5. Testes de integração com Jest contra PostgreSQL e Mailcatcher em contêineres (test.yaml). Não há testes unitários isolados nem medição de cobertura nesta fase.
6. Segurança: secretlint procura segredos no código (bloqueante). SCA (npm audit) e SAST não fazem parte do pipeline nesta fase.
7. Empacotamento: realizado pela Vercel no Preview Deploy; não há registro de imagens de contêiner.

5.2 Política de Pull Request e Revisão de Código

- Toda mudança na main é feita por Pull Request. Por ser um projeto individual, não há segundo revisor: a revisão é feita pelo próprio autor antes do merge, e os checks automáticos atuam como segundo par de olhos.
- PRs devem ser pequenos e de vida curta, para reduzir conflitos e facilitar revisão.
- Merge somente com os checks verdes: Prettier, ESlint, commitlint, secretlint e Jest Ubuntu. Como ainda não há proteção de branch configurada, essa regra é cumprida por disciplina do desenvolvedor, sem bloqueio automático do GitHub.

5.3 Critérios de Qualidade (Quality Gate)

Critérios objetivos para merge na main: 100% dos testes de integração aprovados; zero erros de Prettier e ESLint; 100% dos commits no padrão Conventional Commits; zero segredos detectados pelo secretlint. Não fazem parte do gate nesta fase: cobertura de testes (não medida) e vulnerabilidades em dependências (sem SCA no pipeline). Na data deste plano, uma verificação manual com npm audit aponta 5 vulnerabilidades em dependências de produção (1 crítica, 3 altas e 1 moderada), aceitas temporariamente por se tratar de projeto de teste em desenvolvimento (ver seção 11).

6. Entrega e Implantação Contínua (CD)

No CTN, a implantação é feita pela integração Git da Vercel: cada commit é construído e implantado automaticamente — em Preview, durante o PR, e em Produção, após o merge na main. A Vercel executa um build de produção a partir do mesmo commit validado no PR.

6.1 Etapas do Pipeline de CD

1. Deploy automático em Homologação: Preview Deploy da Vercel a cada PR, com banco de homologação separado no Neon.
2. Testes de aceitação/E2E: não automatizados nesta fase; quando a mudança exige, os endpoints são validados manualmente no Preview.
3. Gate para produção: PR com todos os checks verdes (seção 5.2), verificado pelo desenvolvedor antes do merge. Mudanças listadas na seção 8.2 exigem registro no CCM antes do merge.
4. Deploy em produção: automático pela Vercel após o merge na main (estratégia na seção 6.2).
5. Pós-deploy: aplicação das migrations pendentes via POST /api/v1/migrations (manual) e verificação de saúde via GET /api/v1/status.
6. Baseline e notas de versão: ainda não automatizadas; cada entrega é identificada pelo PR, pelo commit de merge e pelo deploy correspondente na Vercel (seção 9).

6.2 Estratégias de Implantação

Estratégia	Descrição e uso no CTN
Deploy atômico (Vercel) — adotada	A nova versão é construída por completo e só então passa a receber todo o tráfego; a versão anterior permanece disponível para retorno imediato, de forma semelhante ao Blue-Green.
Rolling update — não aplicável	Não há instâncias administradas pelo projeto; a execução é serverless e gerenciada pela Vercel.
Canary Release — não adotada	Não utilizada nesta fase, por não haver usuários reais nem volume de tráfego que a justifique.
Feature Flags — não adotadas	Não utilizadas nesta fase; funcionalidades são entregues completas por PR.

6.3 Rollback e Recuperação

Toda implantação da aplicação é reversível: pelo Instant Rollback da Vercel, que volta a servir um deploy de produção anterior sem novo build (respeitando os limites do plano da Vercel), ou por git revert em um PR, que gera um novo deploy. Limitação: as migrations do projeto têm apenas direção up (exports.down = false), então o esquema do banco não é revertido automaticamente; uma reversão de esquema exige nova migration corretiva e aciona o CCM.

7. Infraestrutura como Código (IaC)

O CTN não utiliza IaC declarativa (Terraform, Pulumi): a infraestrutura de Homologação e Produção é gerenciada pelas plataformas Vercel e Neon e configurada pelos painéis. São versionados como código: o ambiente local (infra/compose.yaml), o esquema do banco (migrations), e os pipelines (.github/workflows/).

7.1 Versionamento de Infraestrutura

- Alterações em infra/compose.yaml, migrations e workflows seguem o mesmo fluxo de PR + checks usado para a aplicação.
- Configurações dos painéis da Vercel e do Neon (variáveis de ambiente, integração Git, branches do banco) não possuem state file; alterações relevantes são registradas em Issue com a label ccm (seção 8.2). A adoção de IaC para essas plataformas é uma pendência (seção 11).

7.2 Ambientes Efêmeros

Adotado: cada Pull Request gera automaticamente um Preview Deploy da Vercel, com URL própria, para validação isolada da mudança, conectado ao banco de homologação no Neon. O Preview deixa de ser atualizado após o merge ou fechamento do PR.

8. Controle de Configuração e Mudança

8.1 Processo de Solicitação de Mudança

Mudanças de rotina seguem o fluxo padrão: Issue no GitHub (dispensável para mudanças triviais) → branch feature/… → commits no padrão Conventional Commits (Commitizen + commitlint) → Pull Request para a main → checks de CI e Preview Deploy → merge → deploy em produção pela Vercel → migrations, se houver. O histórico de PRs, checks e deploys constitui o registro formal e rastreável da mudança.

8.2 Comitê de Controle de Mudança (CCM)

O CCM é composto unicamente por Paulo Guilherme. Em situações rotineiras, o fluxo de PR com checks automáticos substitui o comitê. O CCM é acionado nos seguintes casos:

- Migration que altere ou remova colunas ou tabelas com dados em produção.
- Alteração de variáveis de ambiente/segredos na Vercel ou da configuração do Neon.
- Alteração ou remoção dos workflows de CI ou dos checks executados nos PRs.
- Hotfix que precise contornar o fluxo padrão.
- Atualização MAJOR de dependência (ex.: nodemailer 7 → 10).

Registro: cada acionamento gera uma Issue com a label ccm contendo a descrição da mudança, o impacto, o plano de rollback e a decisão tomada, vinculada ao PR correspondente. A label é criada no primeiro acionamento do CCM.

9. Baselines e Releases

9.1 Geração Automática de Baseline

A geração automática de baseline ainda não está implantada no CTN. Hoje, cada merge na main funciona como uma baseline implícita: o commit de merge, o PR correspondente, o deploy de produção da Vercel desse commit e o estado das migrations (tabela pgmigrations) identificam a versão em produção. O package.json declara a versão 1.0.0, mas ainda não há tags no repositório. A automação de tags SemVer e GitHub Releases a partir dos Conventional Commits está planejada (seção 11).

9.2 Notas de Versão (Release Notes)

Não há notas de versão formais nesta fase. O histórico de mudanças pode ser reconstruído a partir dos Conventional Commits e das descrições dos PRs, que já são padronizados. A geração automática das notas junto às releases está planejada (seção 11).

10. Estimativa do Status de Configuração

10.1 Monitoramento e Observabilidade

Monitoramento compatível com um projeto de teste: o endpoint GET /api/v1/status funciona como health check (versão do PostgreSQL, conexões máximas e abertas), complementado pelos logs de build e de funções da Vercel e pelo painel do Neon. Não há alertas automáticos nesta fase.

10.2 Métricas DORA

Métrica	Descrição e forma de medição no CTN
Deployment Frequency	Frequência de implantação em produção: número de deploys de produção na Vercel (merges na main) por semana.
Lead Time for Changes	Tempo entre o primeiro commit do PR e o deploy de produção correspondente (datas do PR e do deploy na Vercel).
Change Failure Rate	Percentual de deploys de produção seguidos de rollback, revert ou fix corretivo.
Mean Time to Restore (MTTR)	Tempo entre a detecção de uma falha (health check ou Issue) e a restauração do serviço (rollback ou correção em produção).

10.3 Relatórios e Auditorias

O histórico de PRs e checks do GitHub Actions, o histórico de deploys da Vercel, as Issues com a label ccm e a tabela pgmigrations constituem os insumos de auditoria de GCS, dispensando relatórios manuais.

11. Marcos

Marcos do esforço de GCS/DevOps do CTN e evidências verificáveis no repositório:

Marco	Situação	Evidência
Controle de versão com PRs e Conventional Commits	Concluído	Histórico de commits e PRs (#1 a #48); commitlint e Husky
Pipeline de CI (lint, commits, segredos, testes)	Concluído	linting.yaml e test.yaml na aba Actions
Deploy contínuo na Vercel com Neon (Homologação e Produção)	Concluído	Deploys e Previews no painel da Vercel
Proteção da main e checks obrigatórios	Planejado	—
SCA no pipeline de CI (npm audit)	Planejado	—
Release automatizada (tags SemVer e GitHub Releases) — baseline v1.0.0	Planejado	—
Correção das vulnerabilidades conhecidas	Planejado	next 14.2.5 → 14.2.35; avaliar nodemailer 7 → 10 (CCM)
Proteção do endpoint de migrations e migrations automáticas pós-deploy	Planejado	—
Atualização do Node.js 18 (EOL) para versão LTS suportada	Planejado	—
Avaliação de IaC para Vercel/Neon e de SAST	Planejado	—

Riscos conhecidos e aceitos temporariamente (projeto de teste, em desenvolvimento, sem usuários reais):

- O endpoint /api/v1/migrations não exige autenticação: qualquer pessoa pode consultar e aplicar as migrations pendentes em produção. O risco é limitado, pois apenas migrations já versionadas no repositório podem ser executadas.
- 5 vulnerabilidades conhecidas em dependências de produção (1 crítica, 3 altas, 1 moderada), identificadas pelo npm audit.

- Node.js 18 (lts/hydrogen) está fora de suporte (EOL).
- A main não possui regra de proteção: o fluxo de PR com checks verdes depende da disciplina do desenvolvedor.
- Não há tags nem releases: a versão em produção é identificada pelo commit e pelo deploy da Vercel.
- Ausência de medição de cobertura, testes E2E, SAST, SCA no pipeline e IaC declarativa.
- Revisão de código sem segundo revisor, por se tratar de projeto individual.

12. Treinamento e Recursos

Pessoal: um desenvolvedor (Paulo Guilherme), acumulando todos os papéis. Ferramentas e recursos: contas gratuitas no GitHub (repositório e Actions), na Vercel e no Neon; Docker e Node.js no ambiente local. Treinamento: o curso.dev, base do projeto, e a documentação oficial de GitHub Actions, Conventional Commits, Vercel e Neon. Estudos complementares previstos: automação de releases (semantic-release), IaC (Terraform), observabilidade e análise de segurança (SAST e SCA).

13. Controle de Software de Subcontratados, Fornecedores e Dependências de Terceiros

Bibliotecas e serviços de terceiros integrados ao produto também são itens de configuração e devem ser versionados explicitamente (arquivo de dependências travado — lockfile).

- Versões travadas: o package.json declara versões exatas (sem ^ ou ~) e o package-lock.json trava a árvore completa; a instalação no CI é sempre feita com npm ci.
- Verificação de vulnerabilidades em dependências feita manualmente com npm audit; a integração ao pipeline de CI está planejada (seção 11).
- Atualizações de dependências manuais e periódicas (npm outdated), cada uma em PR próprio com commit do tipo build; atualizações MAJOR passam pelo CCM. Dependabot/Renovate não são adotados nesta fase.

Não há software desenvolvido por subcontratados ou fornecedores externos. Os serviços de terceiros utilizados — GitHub (repositório e CI), Vercel (hospedagem e deploy) e Neon (PostgreSQL gerenciado) — são tratados como dependências de plataforma: suas configurações relevantes estão documentadas neste plano e alterações passam pelo CCM (seção 8.2).

14. Quadro-Resumo do Pipeline CI/CD

Resumo do fluxo padrão, do commit à produção:

Etapa	Descrição	Gatilho
CI	Prettier, ESLint, commitlint, secretlint e Jest.	Abertura/atualização de Pull Request para a main
CD — Homologação	Preview Deploy da Vercel com banco de homologação no Neon; validação manual quando necessário.	Abertura/atualização de Pull Request (em paralelo ao CI)

Etapa	Descrição	Gatilho
Gate / Merge	Integração na main por PR, verificada pelo desenvolvedor; mudanças da seção 8.2 exigem registro no CCM.	Todos os checks verdes
CD — Produção	Deploy atômico pela Vercel; migrations via POST /api/v1/migrations; health check em /api/v1/status.	Merge na main
Baseline / Release	Ainda não automatizada: PR, commit de merge e deploy da Vercel identificam cada versão; tags SemVer planejadas.	Merge na main

Nota: *este plano adapta o template ao contexto real do CTN — projeto individual, de teste e em desenvolvimento —, mantendo os quatro pilares da GCS (identificação, controle de versão, controle de mudanças e auditoria/status) suportados por automação sempre que viável.*